Version 1 - april ’26

PON Workshop

R Shiny

Build Dashboards and Apps using RShiny

STAN WIJN

Questions or remarks? Email to: stan.wijn@medipanalytics.com.

Copyright © 2026 Stan R.W. Wijn.

Content

Workshop program 3
Introduction 4
General introduction to the course 4
Background information RShiny 5
When should you build an RShiny app? 5
When is RShiny less suitable? 6
Shiny Basics: Client vs Server 6
Assignment: Example of simple app 8
Assignment: Create first Shiny app 10
User interface 14
Layout 14
Assignment: User interface 16
Different types of input 17
Different types of output 21
Assignment: Create your own resume in R Shiny 23
Assignment: Build a Dynamic Resume Generator in RShiny 24
Assignment: Survey result 32
Answers to questions 35

Workshop program

Below you will find a provisional schedule for the workshop. Please note that this schedule may be subject to change depending on the speed at which certain parts are completed.

Time indication	Description
15:00	Introduction
15:05	Background information RShiny
15:20	Simple Shiny app example
15:30	**Assignment**: Create a simple RShiny app
15:45	RShiny examples
15:55	CSS and styling
16:10	**Assignment**: PhD Resume application
16:45	Hosting
16:55	Closing remarks and questions
17:00	Drinks

IBAN: NL36 RABO 0198 0656 47
KVK: 93607644
BTW: NL866466265B01

Medip Analytics B.V.
Nijmegen
t +31 (0)24-7009837
e info@medipanalytics.com
w www.medipanalytics.com

Introduction

General introduction to the course

Good day and welcome to the documentation for day 1 of the RShiny course. We'll start with an introduction to Shiny and why it might be relevant in your workflow.

Next, we'll discuss the situations in which it might be helpful to build a Shiny app and explain the differences between the client and server-side aspects of Shiny. We'll also discuss the input and output definitions required for building a Shiny app.

The focus of this day will be on building your first Shiny app, and we will focus on defining both the input and output, as well as improving the layout of the app.

We hope this documentation will help you prepare for and understand the content of this first day of the course. Thank you for your interest in learning RShiny, and we look forward to your participation in this course.

My name is Dr. Stan Wijn, and I will teach this course. My background is in Biomedical Sciences with a specialization in health economics & health technology assessment (HTA). After obtaining my master's degree, I started working as a Medical Technology Advisor at MedValue (spin-off Radboudumc). In 2018 I started as a PhD student in the Evidence based surgery group led by Prof. Rovers. During my PhD, I evaluated two common orthopedic treatments for the degenerative knee. Most of my analyses consisted of existing patient data and I gained knowledge of statistical methodology, data science and programming.

To get away from my computer every now and then, I was also involved in teaching several bachelor's and master's courses (Medicine & Biomedical Sciences) and I was co-coordinator/lecturer of the master's course "Advanced modelling in economic evaluation". After completing my teaching portfolio, I received my University Basic Qualification (UBK). From 2020 to 2021, I followed the course 'AI for Health', organized by Jheronimus Academy of Data Science, Radboudumc and Radboud University, to get an overview of the possibilities of AI for healthcare.

During my research, he used a lot of study data from randomized studies and cohort studies and during my PhD he specialized in displaying these effects in Shiny apps (and published several times). As a Shiny developer, I am employed in 2021 as CTO and co-founder of "Medip Analytics" where I developed Shiny applications for Biotech and Pharma companies to show the added value of their technology in terms of costs and patient quality of life.

Background information RShiny

RShiny is a web application framework for R, a powerful programming language widely used in statistics, data analysis, and machine learning. RShiny allows users to develop interactive web applications that use R functionality, and make them accessible to others via a web browser. One of the biggest advantages of RShiny is the ability to easily visualize data and share analytics with others. For example, RShiny allows you to build complex data visualizations and interactive dashboards, which enable others to explore data and make better decisions based on that data.

RShiny applications consist of two main components: the user interface (UI) and the server. The UI contains the visual elements of the application, such as buttons, text boxes, and charts, and the server contains the logic needed to process the data and generate the output that the user sees.

Another advantage of RShiny is that it is open source and has an active community of developers, which means that there is a lot of online documentation and support available. This makes it simple for both novice and experienced developers to learn and improve their skills in RShiny.

At a time when data is becoming an increasingly important part of our daily lives, RShiny offers a powerful way to visualize and analyze this data. Whether you're a data scientist, data analyst, policymaker, or student, RShiny provides the tools you need to present and share data with others in an intuitive and interactive way.

When should you build an RShiny app?

There are several scenarios in which building a Shiny app can make sense. Below are some examples of situations where using a Shiny app can be beneficial:

1. Data analysis and visualization: If you've collected data and want to present it in an intuitive and interactive way, building a Shiny app can help you visualize this data and share it with others. For example, you can build an interactive dashboard that allows users to explore and filter data based on their own needs.

2. Research and experimentation: If you're a scientist or researcher, building a Shiny app can help you document your experiments and present the results in a structured way. For example, you can build an app that allows you to change various experimental parameters and visualize the results in real-time.

3. Presentations and demonstrations: If you need to give a presentation or give a demonstration of your work, building a Shiny app can help you to present your ideas in a way that is visually appealing and easy to understand. For example, you can build an interactive presentation that allows you to explore and demonstrate different aspects of your work.

4. Product development: If you have a business and you want to develop prototypes quickly and efficiently, then building a Shiny app can help you experiment with different ideas quickly and easily. For example, you can build a prototype of a new product and test it with customers before deciding to develop it

So, building a Shiny app can be helpful in a variety of situations where presenting and sharing data and analytics is important. It provides an interactive and intuitive way to visualize and explore data, which can help make better decisions and communicate ideas and results with others.

When is RShiny less suitable?

While RShiny is a powerful framework for developing web applications with R-functionality, there are also situations where it may be less suitable. Below are some examples of when RShiny might not be the best choice:

1. Large datasets: If you are dealing with very large datasets, RShiny may be less suitable for building applications. Since RShiny works within the R environment, working with large datasets can be challenging due to the performance and memory limitations of R. In such cases, it may be better to look at other tools such as Spark or Hadoop to work with large datasets.

2. Real-time processing: If you need a web application that requires real-time processing and fast response times, then RShiny may not be the best choice. Because RShiny works with the R language and server-side processing, it can sometimes be slower to respond to client-side requests. In such cases, it may be better to look at other tools specifically designed for real-time processing, such as Node.js or Python Flask.

3. Mobile applications: If you want to develop a mobile application, then RShiny is probably not the best choice. While RShiny can develop web applications that can be accessed on mobile devices, it is not designed for mobile development. For mobile development, there are other frameworks designed specifically for building native mobile apps, such as React Native or Flutter.

4. More complex applications: If you want to develop a highly complex web application with a large number of different features and integrations, RShiny may not be the best choice. While RShiny offers a lot of functionality, it is not as flexible as other web application frameworks such as Django or Ruby on Rails. In such cases, it may be better to use a different framework that better suits the requirements of the application.

It's important to carefully consider whether RShiny is the best option for your specific requirements and to look into other tools and frameworks as needed.

During this course you can use the Shiny cheat sheet. This provides additional background information and a quick overview: https://rstudio.github.io/cheatsheets/shiny.pdf

Shiny Basics: Client vs Server

RShiny is a web application framework designed to make R functionality accessible through a web browser. The framework works on the basis of a client-server model, where the user interface (UI) is displayed on the client-side and the server-side is responsible for processing the data and generating the output. The client-side of RShiny is responsible for displaying the UI of the application. This might include buttons, text boxes, and charts that appear in a web browser. The UI is presented in HTML, CSS, and JavaScript and can be customized to suit the user's needs.

The server-side of RShiny is responsible for processing the data and generating the output. The server receives input from the client-side, processes this input using R functionality, and then generates the output that is displayed to the user. The server is programmed in R and can use the many R packages and functions available for data processing and statistical analysis. A key component of RShiny is the reactive programming paradigm. Reactive programming uses reactive resources to automatically trigger events when the resources are updated. This allows RShiny to dynamically generate the output based on the input provided by the user.

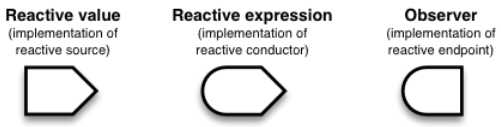


Figure 1: Reactive expressions consist of three parts: Reactive value, Reactive expressions and Observers. Source: shiny.rstudio.com

When working with RShiny, you define the input and output objects that you want to use in the application. Input objects are the elements that the user can manipulate, such as buttons and input fields, while output objects are the elements that are generated from the input, such as graphs and text fields. You then define the reactive expressions that connect the input and output objects and determine the logic for generating the output.

An important part of RShiny's server-side is the session. A session is a unique connection between the server and the client, which opens when the user opens the application and is closed when the user leaves the application. The session is responsible for tracking the state of the application and managing the reactive expressions.

Assignment: Example of simple app

For example, a simple RShiny app that shows the basic functionality of the framework could be an application that enters a number and displays the sum of that number with itself. Below is the code for such an app.

Note the following, and try to understand how this example works:

1. It is noticeable that a Shiny app consists of 3 parts: ui, server, and the run command.
2. The UI has 1 input field to enter a number (**numericInput**) and 1 output field (**textOutput**) that displays text.
3. The server has 2 elements (input and output).
4. The server generated text (**renderText**) and doubled the input.

```
# Load the Shiny library
library(shiny)

# Define the UI of the app
ui <- fluidPage(
  # Define the input field
  numericInput(inputId = "number", label = "Enter a number:", value = 0),

  # Define the output field
  textOutput(outputId = "result")
)

# Define the app's server
server <- function(input, output) {
  # Define the reactive expression that generates the output
  output$result <- renderText({
    input$number + input$number
  })
}

# Run the app
shinyApp(ui, server)
```

In this app, an input field is defined with an inputId of "number". Next, an output field is defined with an outputId of "result". The server function defines the reactive expression that generates the output. This reactive expression adds the number entered in the input field to itself and returns the result as text.

When the app is running with *shinyApp(ui, server),* the UI of the app is displayed in a web browser and the user can enter a number in the input field. Once the user enters the number,

the result is automatically generated and displayed in the output field. This is a simple example of how RShiny can be used to connect input and output objects and generate dynamic output based on user input.

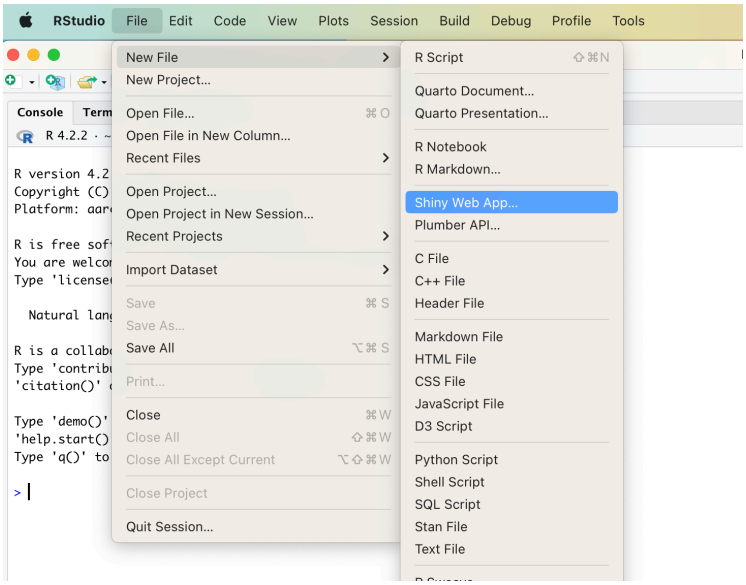
Questions:

1. What is the input ID of the input field in this Shiny app?
2. What is the label of the input field in this Shiny app?
3. What type of input field is used in this Shiny app?
4. What is the function of the renderText() method in the server function?
5. What is the outcome of this Shiny app if the user enters the number 5 in the input field?

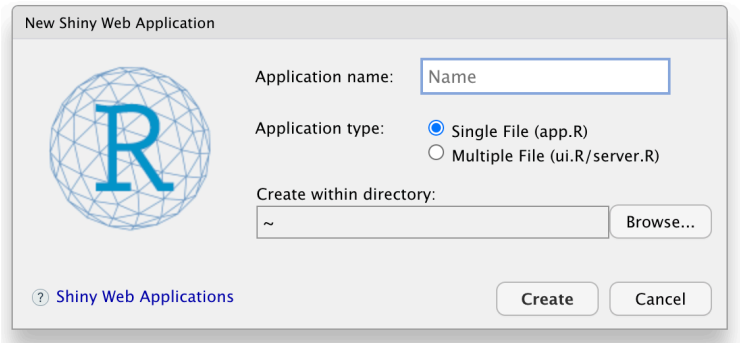
Assignment: Create first Shiny app

Often, just starting is the best way to learn. We're going to start by building a basic Shiny app. For each project in R, it is always a good practice to create a separate project for each topic. Create these.

Creating a first Shiny app in R-Studio is quite easy and can be done via the file menu:



If you select "Shiny Web App..." The next screen pops up:



Give your application a name. For now, you can leave all other settings.

Note: For application type, you can choose between Single File and Multiple File. We will come back to this on day 3. For now it is good to know that with the Single file the UI and Server component are in 1 file ("app. R"). With the multiple file type, RStudio creates a separate file for the UI and a separate file for the server. This is useful when applications become complex.

If you haven't installed Shiny yet, you can install it using:

```
install.packages("shiny")
```

If you already have Shiny installed, use packageVersion("shiny") to check if you have version 1.5.0 or higher.

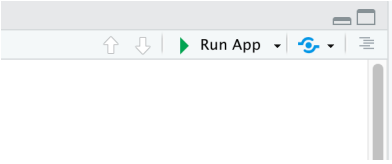
Then load into your current R session:

```
library(shiny)
```

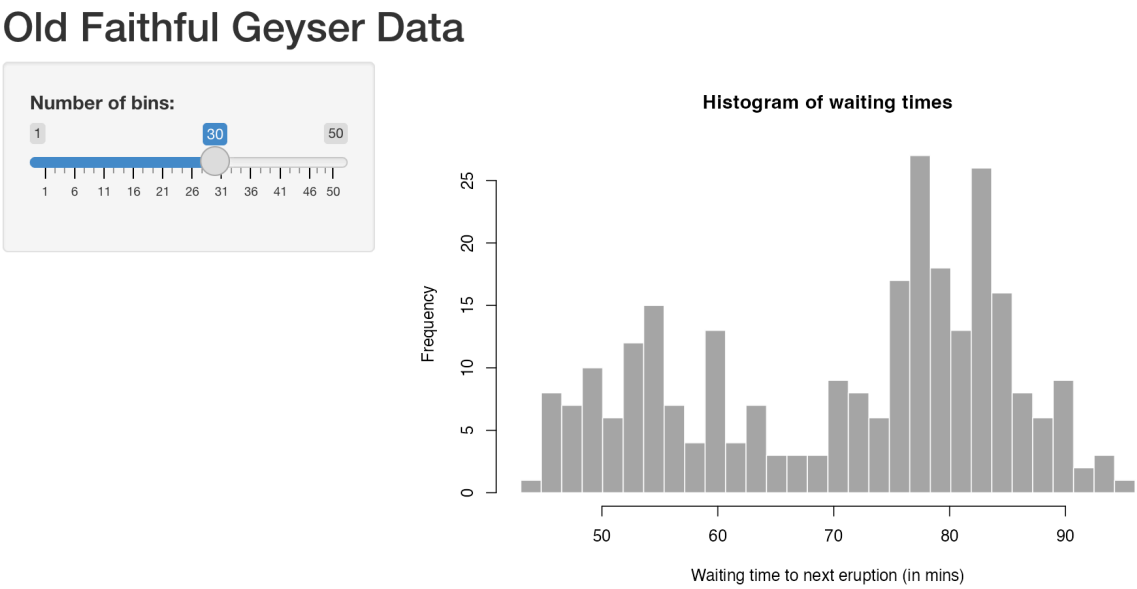
An "app. R" file with the following code:

```
library(shiny)

# Define UI for application that draws a histogram
ui <- fluidPage(
    # Application title
    titlePanel("Old Faithful Geyser Data"),

    # Sidebar with a slider input for number of bins
    sidebarLayout(
        sidebarPanel(
            sliderInput("bins",
                        "Number of bins:",
                        min = 1,
                        max = 50,
                        value = 30)
        ),

        # Show a plot of the generated distribution
        mainPanel(
           plotOutput("distPlot")
        )
    )
)

# Define server logic required to draw a histogram
server <- function(input, output) {
    output$distPlot <- renderPlot({
        # generate bins based on input$bins from ui. R
        x <- faithful[, 2]
        bins <- seq(min(x), max(x), length.out = input$bins + 1)

        # draw the histogram with the specified number of bins
        hist(x, breaks = bins, col = 'darkgray', border = 'white',
             xlab = 'Waiting time to next eruption (in mins)',
             main = 'Histogram of waiting times')
    })
}

# Run the application
shinyApp(ui = ui, server = server)
```

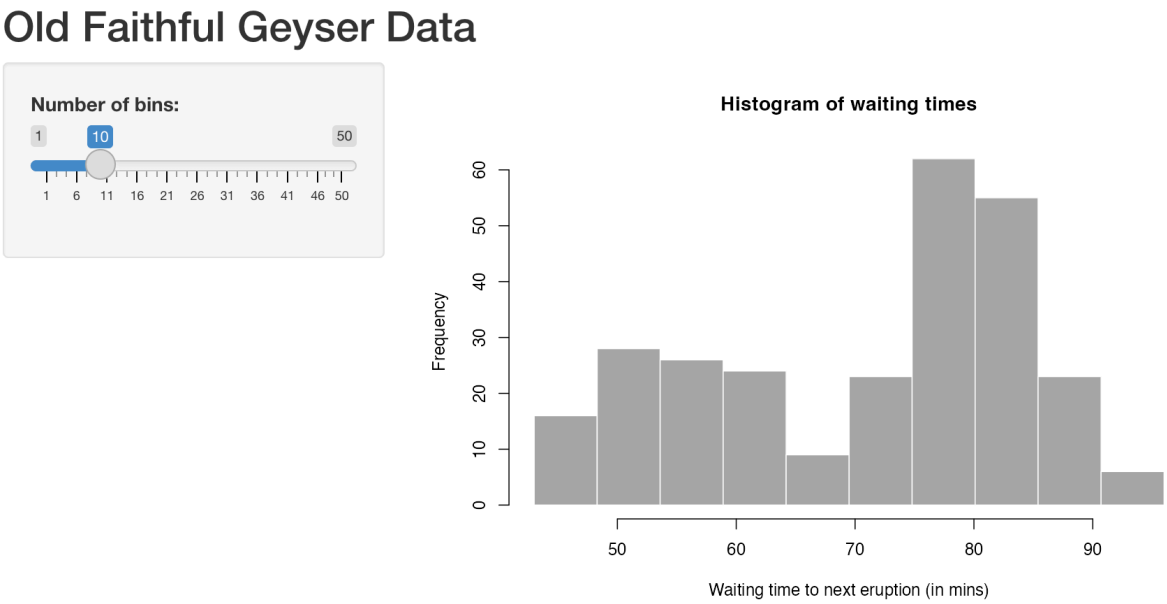
Before we start working on this app, we first click on "Run App", at the top right of the screen:



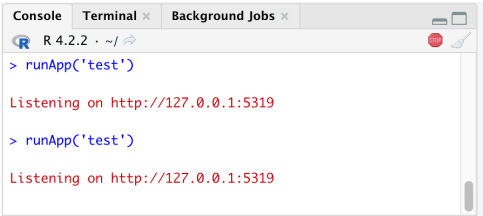
This is how we start the Shiny app. A new screen will open with the Shiny application.



By sliding the slider we change the number of 'bars' that are visible in the image. It is noticeable that the image changes immediately as soon as the slider is moved.



While the application is running, you will see that the console displays the following message:



While the Shiny app is running, you won't be able to send commands in the console. This is indicated by the red stop button in the top right corner. The console will display the IP address on

which the Shiny app is active. If you enter this address in your own internet browser (such as Chrome, Safari or Edge), you can also view the Shiny app.

The Shiny app cannot be accessed by anyone else outside of your computer.

You can close the Shiny app by one of these options:

-Close the application screen (which starts up when you clicked on 'Run app')
-Click on the red stop button
-By selecting the console and pressing ctrl+c.

Questions:

1. What is the minimum number of bins that can be selected using the slider in this Shiny app?
2. What is the maximum number of bins that can be selected using the slider in this Shiny app?
3. What is the slider's 'id'? Where does it appear in this shiny app?
4. What is plotted in the mainPanel of this Shiny app?
5. What happens if the user sets the number of bins to 10 in the slider of this Shiny app? Try to understand the steps in the UI and Server.
6. Now we are going to adjust the Shiny app. Create a second slider under the current slider with the following properties: id = bins2, label = "Number of bins:", min=1, max =6, value =4.
7. In the server, create a new render plot to create a new graph. In this graph we show the contents of the first column of the faithful dataset (faithful[,1]). By means of the slider that we created in step 6, we are going to adjust the bins of this graph.
8. Tips: Don't forget to add the graph to the mainPanel. Make sure that input and output IDs are unique.
9. Extra: change the input to something other than a sliderInput, such as numericInput.

User interface

Layout

The User interface (U.I.) consists of 3 parts: Layout, Input, and Output. The layout determines how the app looks. How is the app laid out? Is there a sidebar? Does the app have a title? What color are the elements in the app? Once the layout is created, it can be filled with input and output elements, such as a slider and a figure.

You can make the layout of a Shiny app as complex as desired by extending the standard functionality with HTML, CSS and Javascript. This will be discussed in the second and third course day (depending on your wishes and background)

The layout of an RShiny app is an important aspect of the user experience. A well-designed layout can help ensure that your app is user-friendly and intuitive for the user. In this guide, we'll discuss some of the key components of a Shiny app's layout.

- The **fluidPage()** function

fluidPage() is the function used to define the layout of your Shiny app. It is one of the most used features and offers a flexible and responsive layout. With **fluidPage()** you can position different types of input and output objects, such as text, graphs, tables, buttons and more.

```
fluidPage(
   titlePanel("Title of the app"),
   sidebarLayout(
    sidebarPanel(
    # Define input objects
    ),
    mainPanel(
    # Define output objects
    )
   )
)
```

- The **titlePanel()** function

titlePanel() is used to define the title of your app and is usually placed at the top of the page. This is the first step in creating a professional look for your Shiny app.

```
titlePanel("Title of the app")
```

- The **sidebarPanel()** function

sidebarPanel() is used to define input objects that are placed on the side of the page. This is useful when you want to group a number of input objects and place them separately from the output objects.

```
sidebarPanel( # Define input objects )
```

- The **mainPanel()** function

mainPanel() is used to define your app's output objects. This is the space where your app's results are displayed.

```
mainPanel( # Define output objects )
```

- The **navbarPage()** function

navbarPage() is used to create a navigation bar at the top of your app. This is useful when you have different sections or pages that you want to group together and make them easily accessible to the user.

```
navbarPage(
 "Title of the app",
 tabPanel("Tab 1",
 # Define input objects and output objects
 ),
 tabPanel("Tab 2",
 # Define input objects and output objects
 )
)
```

These are some of the most important components of a Shiny app's layout. By using these components and combining different input and output objects, you can create an attractive and user-friendly interface for your app.

Assignment: User interface

To build a Shiny app, combine the various elements we've discussed to create an intuitive and attractive user interface. Here are some suggestions to help you build your own Shiny app.

- Use a crisp and straightforward title

A title is the first thing your users will see when they open your app. Make sure your app's title is clear and informative so that the user knows what to expect right away.

- Choose a suitable layout

Choose a layout that suits your specific application and your users. Think carefully about the placement of the input and output objects to create a clear and intuitive user interface.

- Make use of tabs

Tabs can be useful when you want to group different sections or pages in your app. This provides an uncluttered user interface and makes it easy for the user to navigate between the different sections.

- Make use of a navigation bar

A navigation bar can be useful when you want to group different sections or pages in your app. This provides an uncluttered user interface and makes it easy for the user to navigate between the different sections.

- Ensure a consistent design

Make sure your Shiny app has a consistent design. For example, use the same colors, fonts, and font sizes throughout the app to create a professional look.

- Use the widget gallery

Try new widgets from: https://shiny.posit.co/r/gallery/widgets/widget-gallery/

- Test your app

Test your app extensively to ensure that all input and output objects are working correctly and that the user interface is clear and intuitive.

By following these suggestions, you can build a Shiny app that is not only functional but also looks professional and is easy to use for your users.

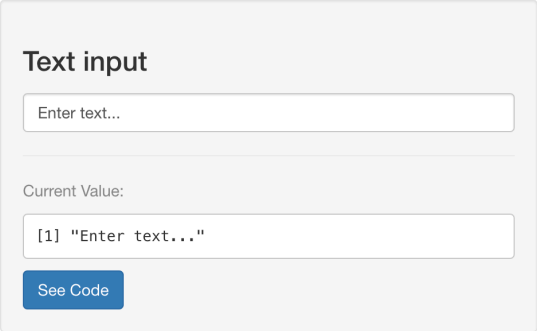
Different types of input

As you can see, there are several ways of input for a shiny app. This input can consist of sliders or numerical input, for example. For example, a slider allows the user to choose a value, which is then displayed in a figure or table.

RShiny provides several types of input that you can use to collect user data and process it in your application. Below are some of the most common types of input available in RShiny.

Text Input

textInput **()** allows you to create a single-line input line, in which the user can type text. You can use the value argument to set a default value for the text.




```
textInput(inputId = "text", label = "Enter text:", value = "Default value")
```

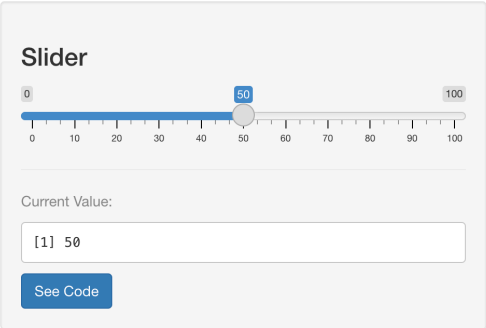
Numeric Input

NumericInput() allows you to create a single-line input rule, in which the user can enter numeric values. You can use the value argument to set a default value for the numeric field.

```
numericInput(inputId = "number", label = "Enter a number:", value = 0)
```

Slider Input

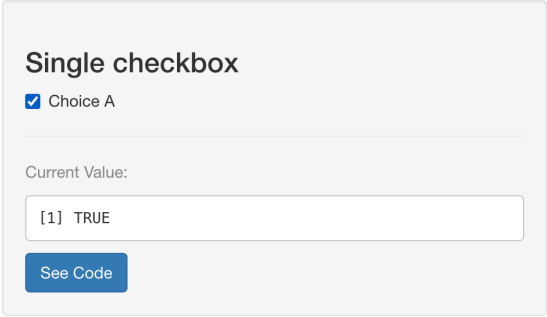
With **sliderInput()** you can create a slider that allows the user to select numeric values within a certain range. You can use the min, max, and value arguments to set the minimum and maximum values of the slider and the default value.




```
sliderInput(inputId = "slider", label = "Select a value:", min = 0, max = 100,
value = 50)
```

Checkbox Input

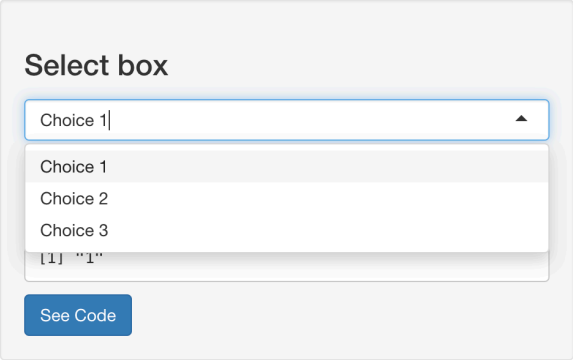
CheckboxInput() allows you to create a single checkbox that allows the user to select a boolean value. You can use the value argument to set the default value.




```
checkboxInput(inputId = "checkbox", label = "Select a value:", value = TRUE)
```

Select Input

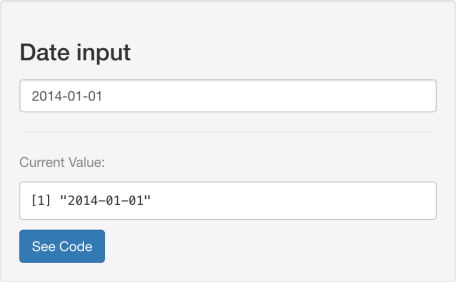
SelectInput **()** allows you to create a dropdown menu that allows the user to select one or more values. You can use the choices argument to set the values in the dropdown menu, and the selected argument to set the default value.




```
selectInput(inputId = "select", label = "Select a value:", choices = c("Value 1",
"Value 2", "Value 3"), selected = "Value 1")
```

Date Input

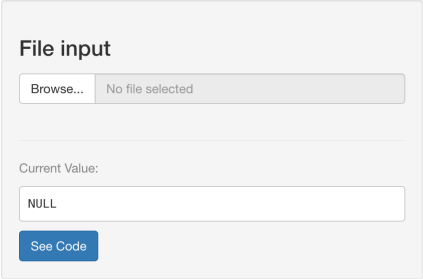
DateInput **()** allows you to create an input rule that allows the user to select a date. You can use the value argument to set the default date.




```
dateInput(inputId = "date", label = "Select a date:", value = Sys.Date())
```

File Input

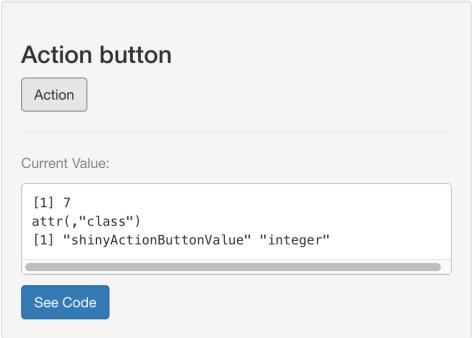
FileInput **()** allows you to create an input rule that allows the user to upload a file. You can use the accept argument to limit the type of file that can be uploaded, and the multiple argument to determine whether multiple files can be uploaded.




```
fileInput(inputId = "file", label = "Upload a file", accept = ".csv")
```

Action Button

ActionButton **()** allows you to create a button that allows the user to initiate an action in the application. When the button is pressed, a value is assigned to a corresponding **reactive()** expression, which triggers the action.




```
actionButton(inputId = "button", label = "Click to start action")
```

Note: from Shiny v1.6.0 onwards, it is recommended to use actionButton in combination with bindEvent() instead of observeEvent() or eventReactive().

These are some of the most common types of input available in RShiny. When building your RShiny application, you can combine and customize these types of input to create a rich and interactive user interface. It's important to remember that depending on your specific application, you might use different types of input. However, RShiny offers a vast selection of input elements, so you can choose what best suits your needs.

For a comprehensive overview of all Shiny input, check out https://shiny.rstudio.com/gallery/widget-gallery.html to try them out live and get code samples. On day 3 of the course we will look at additional packages to expand the range of input widgets.

Different types of output

RShiny offers several types of output that you can use to present results and information to the user. Below are some of the most common types of output available in RShiny.

1. Text Output

textOutput() allows you to display text in your Shiny app. The text can be generated by a **reactive()** expression that is associated with an input object.

```
textOutput(outputId = "text")
```

2. Plot Output **plotOutput()** allows you to display a graph in your Shiny app. The graph can be generated by a **reactive()** expression that is associated with an input object. We will do this in the next assignment.

```
plotOutput(outputId = "plot")
```

3. Table Output **tableOutput()** allows you to display a table in your Shiny app. The table can be generated by a **reactive()** expression that is associated with an input object. We are also going to do this in the extra assignment below.

```
tableOutput(outputId = "table")
```

4. Image Output **imageOutput()** allows you to display images in your Shiny app. The images can be generated by a **reactive()** expression that is linked to an input object.

```
imageOutput(outputId = "image")
```

5. HTML Output **htmlOutput()** allows you to display HTML in your Shiny app. The HTML can be generated by a **reactive()** expression that is attached to an input object.

```
htmlOutput(outputId = "html")
```

6. Download Button downloadButton **()** allows you to create a button that allows the user to download a file. You can use the **filename** argument to set the name of the file to be downloaded, and the **content** argument to set the content of the file to be downloaded.

```
downloadButton(outputId = "download", label = "Download file", filename =
"mijnbestand.txt", content = mycontent)
```

These are some of the most common types of output available in RShiny. When building your RShiny application, you can combine and customize these types of output to present the results to the user in a clear and attractive way. It's important to remember that depending on your specific application, you might use different types of output. However, RShiny offers a vast selection of output elements, so you can choose what best suits your needs.

Assignment: Create your own resume in R Shiny

Objective: Create an R Shiny application that presents a personal resume in a clean two-column layout. The app should include a personal header, profile section, education, skills, a profile picture, and a LinkedIn link.

Steps:

1. Create a new basic Shiny app and run it to confirm that it works.
2. Add a titlePanel() within the fluidPage().
3. Add a personal header below the titlePanel() inside the fluidPage().
4. Split the page into two columns using fluidRow() and column().
5. Use width = 6 for both columns and use h3() and p() elements for section headers and text.
6. Add a profile section.
7. Add an education section using tags$ul() and tags$li() to display bullet points.
8. Add a skills section.
9. Add simple styling to improve the appearance of the resume.
10. Add a profile picture by placing the image in the www folder.
11. Add a LinkedIn link.
12. Run the app and publish it on shinyapps.io.
13. EXTRA: Style it using CSS or use a style that we discussed.
14. EXTRA: Add a downloadable PDF version of the CV.

Assignment: Build a Dynamic Resume Generator in RShiny

Goal: Create a more advanced RShiny application that dynamically generates a resume based on user input. The app should allow users to edit personal details, profile text, skills, and optional publications, and immediately display the updated resume in the main panel. Include a download option so the generated resume can be exported as a text file or PDF-ready content.

Steps:

1. Create a new RShiny app with the standard ui.R and server.R structure.
2. In the ui.R file, create a fluidPage() with a titlePanel() and a sidebarLayout(). Use the sidebarPanel() for user inputs and the mainPanel() for the resume preview.
3. Add input fields in the sidebarPanel() for name, field of study, email address, LinkedIn URL, profile text, and skills. Use suitable Shiny input functions such as textInput(), textAreaInput(), and checkboxInput().
4. Add a checkboxInput() that allows the user to show or hide the publications section.
5. In the mainPanel(), build the resume layout using fluidRow() and column(). Include sections such as Profile, Education, Skills, Experience, Projects, and Publications.
6. In the server.R file, use reactive expressions or renderUI() so that changes in the sidebar inputs are immediately reflected in the resume preview.
7. Use conditionalPanel() or server-side logic to display the Publications section only when the checkbox is selected.
8. Add a profile picture from the www folder and include a clickable LinkedIn link in the generated resume.
9. Add a downloadHandler() so the user can download the generated resume as a text file. Optionally, prepare the output so it can later be converted to PDF.
10. Run the app and verify that all fields update dynamically, the publications section can be toggled on and off, and the download function works correctly.

Suggested learning focus: reactive programming, dynamic UI updates, conditional rendering, user input handling, and downloadable outputs in Shiny.

Code example for simple resume

```
library(shiny)

ui <- fluidPage(
  tags$head(
    tags$style(HTML("
      body {
        font-family: Arial, sans-serif;
        background-color: #f5f5f5;
      }
      .resume-box {
        background: white;
        padding: 25px;
        border-radius: 8px;
        border: 1px solid #d9d9d9;
        max-width: 1000px;
        margin: 0 auto;
      }
      .header-row {
        display: flex;
        align-items: flex-start;
        gap: 20px;
        margin-bottom: 20px;
      }
      .profile-pic {
        width: 110px;
        height: 140px;
        object-fit: cover;
        border: 1px solid #ccc;
      }
      h1 {
        margin-top: 0;
        color: #24479a;
      }
      h2 {
        border-bottom: 1px solid #ccc;
        padding-bottom: 5px;
        margin-top: 20px;
      }
    "))
  ),

  titlePanel("Interactive PhD Resume in Shiny"),

  div(
    class = "resume-box",

    div(
```

```
      class = "header-row",
      tags$img(src = "profile.jpg", class = "profile-pic"),
      div(
        h1("Stan Wijn"),
        p("PhD Student in Biomedical Sciences"),
        p("Email: your.email@university.edu"),
        p(a("LinkedIn profile", href = "https://www.linkedin.com", target = "_blank")),
        p(a("Download PDF resume", href = "#"))
      )
    ),

    hr(),

    fluidRow(
      column(
        width = 6,
        h2("Profile"),
        p("Health technology assessment specialist with interests in epidemiology, statistics,
and biomedical data analysis."),

        h2("Education"),
        tags$ul(
          tags$li(tags$strong("PhD in Biomedical Sciences"), " – University Name, 2023–
Present"),
          tags$li(tags$strong("MSc in Public Health"), " – University Name, 2021"),
          tags$li(tags$strong("BSc in Biomedical Sciences"), " – University Name, 2019")
        ),

        h2("Skills"),
        tags$ul(
          tags$li("R Shiny"),
          tags$li("HTML"),
          tags$li("Deep learning"),
          tags$li("Statistics"),
          tags$li("Data visualization")
        )
      ),

      column(
        width = 6,
        h2("Experience"),
        tags$ul(
          tags$li(
            tags$strong("PhD Researcher"),
            br(),
            "University Department, 2023–Present",
            br(),
            "Developed R Shiny applications, performed statistical analyses, and communicated
findings through dashboards and reports."
```

```
        ),
        tags$li(
          tags$strong("Research Assistant"),
          br(),
          "Research Institute, 2021–2023",
          br(),
          "Supported data cleaning, analysis, and reporting for academic projects."
        )
      ),

      h2("Projects"),
      tags$ul(
        tags$li("Interactive R Shiny dashboard for research data exploration"),
        tags$li("Automated reporting workflow in R"),
        tags$li("Scientific visualizations for academic presentations")
      ),

      h2("Publications"),
      tags$ul(
        tags$li("Smith J, Wijn S, et al. Example title of paper. Journal Name, 2025."),
        tags$li("Conference presentation: Example presentation title, International
Conference, 2024.")
      )
    )
  )
 )
)

server <- function(input, output) {
}

shinyApp(ui = ui, server = server)
```

Example of dynamic resume

```
library(shiny)

ui <- fluidPage(
  tags$head(
    tags$style(HTML("
      body { font-family: Arial, sans-serif; background-color: #f7f7f7; }
      .panel-box {
        background: #f0f0f0;
        padding: 15px;
        border: 1px solid #d8d8d8;
        border-radius: 6px;
      }
      .resume-preview {
        background: white;
        padding: 25px;
        border: 1px solid #d8d8d8;
        border-radius: 8px;
        min-height: 750px;
      }
      .header-wrap {
        display: flex;
        gap: 20px;
        align-items: flex-start;
        margin-bottom: 20px;
      }
      .profile-pic {
        width: 110px;
        height: 110px;
        object-fit: cover;
        border: 1px solid #ccc;
      }
      h2 {
        border-bottom: 1px solid #ccc;
        padding-bottom: 4px;
        margin-top: 18px;
      }
    "))
  ),

  titlePanel("Dynamic Resume Generator in Shiny"),

  sidebarLayout(
    sidebarPanel(
      class = "panel-box",

      textInput("name", "Full name", "Stan Wijn"),
      textInput("title", "Professional title", "PhD Student in Biomedical Sciences"),
      textInput("email", "Email", "your.email@university.edu"),
      textInput("linkedin", "LinkedIn URL", "https://www.linkedin.com"),
      textAreaInput(
        "profile",
        "Profile summary",
        "Health economist with experience in epidemiology, statistics, and R Shiny dashboard development.",
        rows = 4
      ),

      textAreaInput(
        "education",
        "Education (one item per line)",
        "PhD in Biomedical Sciences - University Name, 2023-Present\nMSc in Public Health - University Name, 2021\nBSc
in Biomedical Sciences - University Name, 2019",
```

```
      rows = 5
    ),

    textAreaInput(
      "experience",
      "Experience (one item per line)",
      "PhD Researcher - University Department, 2023-Present\nResearch Assistant - Research Institute, 2021-2023",
      rows = 5
    ),

    textAreaInput(
      "projects",
      "Projects (one item per line)",
      "Interactive R Shiny dashboard for research data exploration\nAutomated reporting workflow in R\nScientific
visualizations for academic presentations",
      rows = 5
    ),

    textAreaInput(
      "skills",
      "Skills (one item per line)",
      "R\nShiny\nHTML\nStatistics\nData visualization",
      rows = 5
    ),

    textAreaInput(
      "publications",
      "Publications (one item per line)",
      "Smith J, Stan Wijn, et al. Example title of paper. Journal Name, 2025.\nConference presentation: Example title,
International Conference, 2024.",
      rows = 4
    ),

    checkboxInput("show_projects", "Show projects", TRUE),
    checkboxInput("show_publications", "Show publications", TRUE),

    downloadButton("download_resume", "Download resume as text")
  ),

  mainPanel(
    div(
      class = "resume-preview",

      div(
        class = "header-wrap",
        tags$img(src = "profile.jpg", class = "profile-pic"),
        div(
          h1(textOutput("out_name", inline = TRUE), style = "color:#24479a;"),
          p(textOutput("out_title", inline = TRUE)),
          p(textOutput("out_email", inline = TRUE)),
          p(uiOutput("out_linkedin"))
        )
      ),

      fluidRow(
        column(
          width = 6,
          h2("Profile"),
          p(textOutput("out_profile", inline = TRUE)),

          h2("Education"),
          uiOutput("out_education"),

          h2("Skills"),
```

```
          uiOutput("out_skills")
        ),
        column(
          width = 6,
          h2("Experience"),
          uiOutput("out_experience"),

          conditionalPanel(
            condition = "input.show_projects == true",
            tagList(
              h2("Projects"),
              uiOutput("out_projects")
            )
          ),

          conditionalPanel(
            condition = "input.show_publications == true",
            tagList(
              h2("Publications"),
              uiOutput("out_publications")
            )
          )
        )
      )
    )
  )
)

server <- function(input, output) {

  to_list <- function(x) {
    items <- unlist(strsplit(x, "\n"))
    items <- trimws(items)
    items[items != ""]
  }

  output$out_name <- renderText({
    input$name
  })

  output$out_title <- renderText({
    input$title
  })

  output$out_email <- renderText({
    paste("Email:", input$email)
  })

  output$out_linkedin <- renderUI({
    a("LinkedIn profile", href = input$linkedin, target = "_blank")
  })

  output$out_profile <- renderText({
    input$profile
  })

  output$out_education <- renderUI({
    tags$ul(
      lapply(to_list(input$education), tags$li)
    )
  })

  output$out_experience <- renderUI({
```

```
  tags$ul(
    lapply(to_list(input$experience), tags$li)
  )
})

output$out_projects <- renderUI({
  tags$ul(
    lapply(to_list(input$projects), tags$li)
  )
})

output$out_skills <- renderUI({
  tags$ul(
    lapply(to_list(input$skills), tags$li)
  )
})

output$out_publications <- renderUI({
  tags$ul(
    lapply(to_list(input$publications), tags$li)
  )
})

output$download_resume <- downloadHandler(
  filename = function() {
    "dynamic_resume.txt"
  },
  content = function(file) {
    lines <- c(
      input$name,
      input$title,
      paste("Email:", input$email),
      paste("LinkedIn:", input$linkedin),
      "",
      "Profile",
      input$profile,
      "",
      "Education",
      paste("-", to_list(input$education), collapse = "\n"),
      "",
      "Experience",
      paste("-", to_list(input$experience), collapse = "\n"),
      "",
      "Skills",
      paste("-", to_list(input$skills), collapse = "\n")
    )

    if (isTRUE(input$show_projects)) {
      lines <- c(
        lines,
        "",
        "Projects",
        paste("-", to_list(input$projects), collapse = "\n")
      )
    }

    if (isTRUE(input$show_publications)) {
      lines <- c(
        lines,
        "",
        "Publications",
        paste("-", to_list(input$publications), collapse = "\n")
      )
    }
```

```
    writeLines(lines, file)
  }
 )
}

shinyApp(ui = ui, server = server)
```

Assignment: Survey result

In this assignment, we are going to put together all the elements we discussed and build a new app. Build a Shiny app that displays the results of a survey. The app should allow the user to choose from a variety of questions and graphically display the answers to each question.

Assume the following input, where id is a unique input and answers are given to 3 questions (q1 to q3).

```
questions <- data.frame(
  id = 1:10,
  q1 = c(4, 2, 2, 3, 9, 3, 3, 5, 6, 2),
  q2 = c(1, 1, 1, 1, 3, 4, 2, 3, 2, 1),
  q3 = c(8, 4, 3, 9, 4, 6, 7, 2, 9, 4)
)
```

Use the following components to build your app:

1. Use **fluidPage()** to define the layout of your app.
2. Use **titlePanel()** to define a title for your app.
3. Use **sidebarPanel()** to position the input objects.
4. Use **mainPanel()** to position the output objects.
5. Add a navigation bar to your app using **navbarPage()**.
6. Make use of **selectInput()** to allow the user to choose a question.
7. Use **plotOutput()** to define the graphical representation of the answer to the question. (Tip: for extra beautiful plot, you can use the ggplot or plotly package). In the 'renderPlot', subset the data first based on the chosen input by the selectInput. Then use this subset to define the x-axis (factor(id) and y-axis (colom that selected) For the selected question in the "selectInput" show the results for each ID (y = ID, x = value).

You should now have a working RShiny app with an input and an output. If not, check the answer in the folder with RShiny code ('Survey assignment. R"). Now we are going to make it a bit more complex.

8. Create an **actionButton()** in the sidebarPanel that the user clicks to confirm the choice of input and generate a new plot.
9. Use "reactive({})" to sort the dataset by the input. Use according to this reactive dataset as input for the plot. Example:

```
  filtered_data <- reactive({
    data.frame(id = id, value = value)
  })
output$chart <- renderPlot({
    ggplot(filtered_data(), AES(x = factor(id), y = value)) +
      geom_bar(stat = "identity")
```

10. Now add a "bindEvent" after the reactive of the filtered_data with a pipe (%>%) to make the filtering dependent on the input (this requires the "magrittr" package) Example:

```
%>% bindEvent(input$submit)
```

Check the answer in the "Survey assignment" button. R"

Additional challenges:

- Create different tabs (using tabsetPanel() and tabPanel()) for different sections of your survey, and use **tableOutput()** to display the results of all questions in a table.
- Place a logo in the bottom right corner of the app:
  - Create a 'www' folder in the project directory
  - Place a picture called 'logo.png' in it
  - Place the logo in the app with

```
img(src='logo.png', align='right')"
```

IBAN: NL36 RABO 0198 0656 47
KVK: 93607644
BTW: NL866466265B01

Medip Analytics B.V.
Nijmegen
t +31 (0)24-7009837
e info@medipanalytics.com
w www.medipanalytics.com

Answers to questions

Assignment Simple Shiny app:

1. What is the input ID of the input field in this Shiny app? **Answer: "number"**
2. What is the label of the input field in this Shiny app? **Answer: "Please enter a number:"**
3. What type of input field is used in this Shiny app? **Answer: NumericInput**
4. What is the function of the renderText() method in the server function? **Answer: It generates the output of the app based on the user's input.**
5. What is the outcome of this Shiny app if the user enters the number 5 in the input field? **Answer: 10 (because the input number is added together twice in the server function).**

Assignment First Shiny app:

1. What is the title of the application in this Shiny app? **Answer: "Old Faithful Geyser Data"**
2. What is the minimum number of bins that can be selected using the slider in this Shiny app? **Answer: 1**
3. What is the maximum number of bins that can be selected using the slider in this Shiny app? **Answer: 50**
4. What is plotted in the mainPanel of this Shiny app? **Answer: A histogram of the waiting times until the next eruption of the Old Faithful geyser.**
5. What happens if the user sets the number of bins to 10 in the slider of this Shiny app? **Answer: The histogram is re-plotted with 10 bins to show the distribution of waiting times until the next eruption of the geyser.**

Assignment Survey app:

```
# Load required libraries
library(shiny)
library(ggplot2)
library(magrittr)


# Create a dataset of survey questions and answers
questions <- data.frame(
  id = 1:10,
  q1 = c(4, 2, 2, 3, 9, 3, 3, 5, 6, 2),
  q2 = c(1, 1, 1, 1, 3, 4, 2, 3, 2, 1),
  q3 = c(8, 4, 3, 9, 4, 6, 7, 2, 9, 4)
)


# Define the User Interface
ui <- fluidPage(
  titlePanel("Survey Results"),
  sidebarLayout(
    sidebarPanel(
      selectInput("question", "Select a question:",
                  choices = c("q1", "q2", "q3")),
      actionButton(inputId = "submit", label = "Send", class = "btn-success")
    ),
    mainPanel(
      plotOutput("chart")
    )
  )
)


# Define the server
server <- function(input, output) {
  # Define a reactive function to filter data based on the selected question and
submit button
  filtered_data <- reactive({
    data.frame(id = questions$id, value = questions[[input$question]])
  }) %>% bindEvent(input$submit)


  # Initialize the chart renderPlot
  output$chart <- renderPlot({
    ggplot(filtered_data(), AES(x = factor(id), y = value)) +
      geom_bar(stat = "identity") +
      coord_flip() +
      labs(title = paste("Results for", input$question), x = "ID", y = "Value")
  })
}


# Run the app
shinyApp(ui, server)
```